

Kalshi Weather Market Analysis and Trading Automation

<https://github.com/DamianlapenavidalURI/kalshi-AI-automated>

Problem & Motivation



Manual market scanning is slow and inconsistent



Hard to evaluate weather context + market quality quickly



Easy to make unsafe decisions without strict guardrails



Need: repeatable, explainable, safe workflow

Project Goals

Weather-focused

Short-horizon
market selection

AI-assisted
analysis

Deterministic
hard rails before
execution

Architecture Overview

1.

Kalshi API + Weather/Data
Sources

2.

Discovery
+
Candidate Filtering

3.

Multi-Agent Reasoning
(Scout, Context, Edge, Critique, Final)

4.

Deterministic Risk + Execution Engine

5.

SQLite + JSON Diagnostics

6.

Streamlit Dashboards
(User / Developer)

Technical Implementation

Language/Runtime

→ Python 3.11+

Storage

→ SQLite

UI

→ Streamlit
→ Altair

APIs

→ Kalshi API (demo/ live)
→ OpenWeather API
→ + Adapters for
Open-Meteo, NWS,
NHC, USGS
→ OpenAI API
→ LangChain API

Generative AI Usage

Role	Model	Task	Duration/Candidate
SCOUT	→ gpt-4o-mini	Fast triage: keep/reject, priority (0-100), short reason	~1-2s
CONTEXT	→ gpt-4o-mini	Evaluate context quality + settlement clarity; return concerns/key points	~2-4s
EDGE	→ gpt-4.1	Estimate directional edge (yes/no), confidence, notes from prices + research	~2-5s
CRITIQUE	→ gpt-4o-mini	Skeptical pass: veto flag, key risks, confidence penalty	~1-3s
FINAL ORCHESTRATOR	→ o4-mini	Final decision (ENTER/SKIP/WAIT/REDUCE_SIZE) + side, size, reasoning	~3-8s

Deterministic Safety & Risk Rails

01

Market must be
open

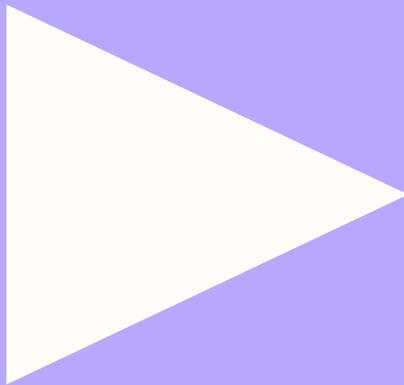
02

Minimum liquidity

03

Anti-repeat
cooldown (same
market/thesis)

MVP DEMO



Outcomes

Challenging



- Finding reasonable deterministic settings to allow betting intents
- Search for weather/like candidates in Kalshi and divide by market type (daily/hourly temp, hurricanes, climate change...)
- Troubleshooting entire pipeline
- Place intents safely and periodically

Worked



- Weather focused candidates
- Handling repeat-thesis behavior without overtrading
- Implementing multiple AI agents
- Keeping AI useful without making safety nondeterministic
- Developer dashboard helped debug drop reasons and stage timing

Did Not Work



- Soccer market automation (too many markets)
- Kalshi UI differs from which tickers the system finds
- Many tickers in demo environment are useless
- The system creates 0 intents most of the times

Next Steps

Functionality



- Fine tune deterministic settings to achieve the most efficient system
- Run the agents in parallel and let the final orchestrator make the final decision
- Set up periodic automation
- Include exit stage where it re-evaluates active bets

Dashboard



- Improve User dashboard
- Add visibility on AI Agents research and decisions
- Add a button that runs the whole system from streamlit app
- Add a historial of runs and outcomes
- Separate demo and live environments



Thank you!